

Consistency

1. Overview

Consistency is one of the most misunderstood concepts in system design because it has **different meanings in different contexts**.

Two major contexts:

1. **ACID Consistency** → Database integrity rules
2. **CAP Consistency** → Distributed systems consistency (linearizability)

In interviews, clearly distinguishing these two gives a strong signal of understanding.

2. Two Meanings of Consistency

A. ACID Consistency (Databases)

In relational databases, consistency means:

After a transaction completes, data remains valid according to defined constraints.

Examples:

- Foreign keys valid
- Unique constraints preserved
- No invalid balances
- Data types respected
- Business invariants maintained

Example:

None

Transfer ₹100:

A = 500

B = 200

After transaction:

A = 400

B = 300

Total still = 700

This is ACID consistency.

B. CAP Consistency (Distributed Systems)

In CAP theorem, consistency usually means **linearizability**.

Meaning:

After a write succeeds, **all clients see the latest value immediately**.

Every read returns most recent committed write.

System behaves like one single up-to-date machine.

3. What Is Linearizability?

Suppose replicated data on 3 nodes.

Before write:

None

$A = 5$

Node1 = 5

Node2 = 5

Node3 = 5

User writes:

None

$A = 10$

Linearizable system:

None

At one logical instant,
all future reads return 10

No stale reads after successful write.

4. Why Consistency Matters

Needed when wrong data is dangerous.

Examples:

- Bank balance
- Stock trading
- Inventory reservation
- OTP verification
- Seat booking
- Order state transitions

Less critical cases may tolerate weaker consistency.

Examples:

- Likes count
- View count
- Recommendations
- Analytics dashboards
- News feed ordering

5. CAP Theorem Refresher

In presence of network partition, distributed systems trade between:

- **C** = Consistency (linearizability)
- **A** = Availability
- **P** = Partition tolerance

Since partitions can happen, usually choose between:

None

CP = Consistent but may reject requests

AP = Available but may return stale data

6. Strong Consistency vs Eventual Consistency

Strong Consistency (Linearizable)

After write:

None

Read immediately returns latest value

Good for correctness.

Eventual Consistency

After write:

None

Some nodes may show old value temporarily

Later all replicas converge

Good for scalability and availability.

7. Example: User Profile Update

User changes profile picture.

Immediately after write:

Strong Consistency

All devices show new photo instantly.

Eventual Consistency

Phone shows new photo, web still old for few seconds.

Later all updated.

8. Example: Bank Balance

User transfers money.

Need:

None

Latest balance visible immediately

Eventual consistency here can cause overdraft / double spend.

So stronger consistency preferred.

9. Why Eventual Consistency Exists

Strong consistency across many nodes requires coordination.

That adds:

- Higher latency
- Lower availability
- More complexity
- Slower writes

So many internet-scale systems use eventual consistency.

10. Databases Often Associated With Models

Systems Favoring Stronger Consistency / Linearizability (config dependent)

- HBase
- MongoDB (majority settings)
- Redis (single primary model)
- Traditional RDBMS

Systems Often Used for Availability / Eventual Consistency

- Cassandra
- Riak

- Dynamo-style systems
- CouchDB

Important note: Many databases allow tunable consistency.

11. Techniques to Achieve Strong Consistency

A. Leader / Primary Writes

All writes go to one leader.

None

Clients → Leader → Replicas

Simpler ordering.

B. Consensus Protocols

Examples:

- Paxos
- Raft
- ZooKeeper style coordination

Used for agreeing on order of writes.

C. Quorum Reads/Writes

If:

None

$$R + W > N$$

Where:

- N = replicas
- R = read quorum
- W = write quorum

Can guarantee latest read in many models.

D. Full Mesh Coordination

All nodes coordinate before serving updates.

Expensive at scale.

12. Techniques for Eventual Consistency

A. Async Replication

Write primary first, replicas later.

B. Event Sourcing

Append events, propagate asynchronously.

C. Distributed Cache Propagation

Cache updated gradually.

D. Gossip Protocol

Nodes periodically exchange state.

E. Random Leader / Peer Sync

Lower cost, weaker guarantees.

13. Example: Product Inventory

Suppose last item in stock.

Eventual Consistency Risk

Two regions both see stock=1 and oversell.

Strong Consistency Solution

Use centralized reservation / quorum write.

14. Trade-offs Table

Property	Strong Consistency	Eventual Consistency
Correctness	High	Temporary stale reads

Availability	Lower during partition	Higher
Latency	Higher	Lower
Complexity	High	Medium
Scale	Harder	Easier

15. Latency Impact

Strong consistency often needs:

None

Cross-node acknowledgements

Leader confirmation

Quorum responses

Adds network RTTs.

Eventual consistency can acknowledge faster.

16. Availability Impact

If leader unreachable:

Strong systems may reject writes.

Eventual systems may still accept local writes and sync later.

17. Interview Requirement Questions

Ask:

1. Can stale reads be tolerated?
2. Is double booking acceptable?
3. How long can inconsistency last?
4. Global multi-region writes needed?
5. Read-heavy or write-heavy workload?
6. SLA for correctness vs uptime?

18. Real-World Use Cases

Strong Consistency Needed

- Payments
- Wallet balances
- Ticket booking
- Password changes
- Inventory reservation

Eventual Consistency Acceptable

- Social likes
- Followers count
- Analytics
- News feed ranking
- Cached profiles

19. Common Interview Answer

Q: Which consistency model would you choose?

Answer:

It depends on business correctness requirements. For balances, bookings, or inventory reservation, I'd use strong consistency. For feeds, counters, and recommendations, eventual consistency is often acceptable because it improves latency, availability, and scalability.

20. Misconceptions to Avoid

✗ “Consistency always means same thing.”

No — ACID and CAP meanings differ.

✗ “NoSQL means inconsistent.”

No — many NoSQL systems support tunable or strong consistency.

✗ “Strong consistency is always best.”

Not if latency and availability are critical.

21. Best Practices

- Use strongest consistency only where needed
- Separate critical vs noncritical data paths
- Use eventual consistency for scale-heavy features
- Define acceptable staleness windows
- Monitor replication lag
- Design idempotent conflict resolution

22. Short Revision Notes

None

Consistency has 2 meanings:

ACID:

Data remains valid after transaction

CAP:

Usually linearizability = latest write visible immediately

Strong consistency:

Correct but slower / less available

Eventual consistency:

Scalable, available, may be stale briefly